Profile Language

The best way to create a profile is to modify an existing one. Several example profiles are available on Github: https://github.com/cobalt-strike/Malleable-C2-Profiles

When you open a profile, here is what you will see:

```
# this is a comment
set global_option "value";

protocol-transaction {
    set local_option "value";

    client {
        # customize client indicators
    }

    server {
        # customize server indicators
    }
}
```

Comments begin with a # and go until the end of the line. The set statement is a way to assign a value to an option. Profiles use { curly braces } to group statements and information together. Statements always end with a semi-colon.

To help all of this make sense, here's a partial profile:

```
http-get {
        set uri "/foobar";
        client {
                metadata {
                        base64;
                        prepend "user=";
                        header "Cookie";
                }
        }
```

This partial profile defines indicators for an HTTP GET transaction. The first statement, set uri, assigns the URI that the client and server will reference during this transaction. This set statement occurs outside of the client and server code blocks because it applies to both of them.

The client block defines indicators for the client that performs an HTTP GET. The client, in this case, is Cobalt Strike's Beacon payload.

When Cobalt Strike's Beacon "phones home" it sends metadata about itself to Cobalt Strike. In this profile, we have to define how this metadata is encoded and sent with our HTTP GET request.

The metadata keyword followed by a group of statements specifies how to transform and embed metadata into our HTTP GET request. The group of statements, following the metadata keyword, is called a data transform.

	Step	Action	Data
0.	Start		metadata
1.	base64	Base64 Encode	bWV0YWRhdGE=
2.	prepend "user="	Prepend String	user=bWV0YWRhdGE=
3.	header "Cookie"	Store in Transaction	

The first statement in our data transform states that we will base64 encode our metadata **[1]**. The second statement, prepend, takes our encoded metadata and prepends the string user= to it **[2]**. Now our transformed metadata is "user=" . base64(metadata). The third statement states we will

store our transformed metadata into a client HTTP header called Cookie **[3]**. That's it.

Both Beacon and its server consume profiles. Here, we've read the profile from the perspective of the Beacon client. The Beacon server will take this same information and interpret it backwards. Let's say our Cobalt Strike web server receives a GET request to the URI /foobar. Now, it wants to extract metadata from the transaction.

	Step	Action	Data
0.	Start		
1.	header "Cookie"	Recover from Transaction	user=bWV0YWRhdGE=
2.	prepend "user="	Remove first 5 characters	bWV0YWRhdGE=
3.	base64	Base64 Decode	metadata

The header statement will tell our server where to recover our transformed metadata from **[1]**. The HTTP server takes care to parse headers from the HTTP client for us. Next, we need to deal with the prepend statement. To recover transformed data, we interpret prepend as remove the first X characters **[2]**, where X is the length of the original string we prepended. Now, all that's left is to interpret the last statement, base64. We used a base64 encode function to transform the metadata before. Now, we use a base64 decode to recover the metadata **[3]**.

We will have the original metadata once the profile interpreter finishes executing each of these inverse statements.

Data Transform Language

A data transform is a sequence of statements that transform and transmit data. The data transform statements are:

Statement	Action	Inverse
append "string"	Append "string"	Remove last LEN("string") characters
base64	Base64 Encode	Base64 Decode
base64url	URL-safe Base64 Encode	URL-safe Base64 Decode
mask	XOR mask w/ random key	XOR mask w/ same random key

Statement	Action	Inverse
netbios	NetBIOS Encode ‘a’	NetBIOS Decode ‘a’
netbiosu	NetBIOS Encode ‘A’	NetBIOS Decode ‘A’
prepend "string"	Prepend "string"	Remove first LEN("string") characters

A data transform is a combination of any number of these statements, in any order. For example, you may choose to netbios encode the data to transmit, prepend some information, and then base64 encode the whole package.

A data transform always ends with a termination statement. You may only use one termination statement in a transform. This statement tells Beacon and its server where in the transaction to store the transformed data.

There are four termination statements.

Statement	What
header "header"	Store data in an HTTP header
parameter "key"	Store data in a URI parameter
print	Send data as transaction body
uri-append	Append to URI

The header termination statement stores transformed data in an HTTP header. The parameter termination statement stores transformed data in an HTTP parameter. This parameter is always sent as part of URI. The print statement sends transformed data in the body of the transaction.

The print statement is the expected termination statement for the http-get.server.output, http- post.server.output, and http-stager.server.output blocks. You may use the header, parameter, print and uri-append termination statements for the other blocks.

If you use a header, parameter, or uri-append termination statement on http-post.client.output, Beacon will chunk its responses to a reasonable length to fit into this part of the transaction.

These blocks and the data they send are described in a later section.

Strings

Beacon's Profile Language allows you to use "strings" in several places. In general, strings are interpreted as-is. However, there are a few special values that you may use in a string:

Value	Special Value
"\n"	Newline character
"\r"	Carriage Return
"\t"	Tab character
"\u####"	A unicode character
"\x##"	A byte (e.g., \x41 = 'A')
"\\"	\

Headers and Parameters

Data transforms are an important part of the indicator customization process. They allow you to dress up data that Beacon must send or receive with each transaction. You may add extraneous indicators to each transaction too.

In an HTTP GET or POST request, these extraneous indicators come in the form of headers or parameters. Use the parameter statement within the client block to add an arbitrary parameter to an HTTP GET or POST transaction.

This code will force Beacon to add ?bar=blah to the /foobar URI when it makes a request.

```
http-get {
    client {
        parameter "bar" "blah";
```

Use the header statement within the client or server blocks to add an arbitrary HTTP header to the client's request or server's response. This header statement adds an indicator to put network security monitoring teams at ease.

```
http-get {
     server {
         header "X-Not-Malware" "I promise!";
```

The Profile Interpreter will Interpret your header and parameter statements In order. That said, the WinINet or WinHTTP (client) and Cobalt Strike web server have the final say about where in the transaction these indicators will appear.

See ***HTTP Host Profiles*** for instructions to include customized headers and parameters for specific host names.

Options

You may configure Beacon's defaults through the profile file. There are two types of options: global and local options. The global options change a global Beacon setting. Local options are transaction specific. You must set local options in the right context. Use the set statement to set an option.

```
set "sleeptime" "1000";
```

Here are a few options:

Option	Context	Default Value	Changes
data_jitter		0	Append random-length string (up to data_jitter value) to http-get and http-post server output.
headers_remove			Comma-separated list of HTTP client headers to remove from Beacon C2

Option	Context	Default Value	Changes
host_stage		true	Host payload for staging over HTTP, HTTPS, or DNS. Required by stagers.
jitter		0	Default jitter factor (0-99%) This property cannot be used when the sleep option is included in the profile.
pipename		msagent_##	Default name of pipe to use for SMB Beacon's peer-to-peer communication. *Each # is replaced with a random hex value.*
pipename_stager		status_##	Name of pipe to use for SMB Beacon's named pipe stager. *Each # is replaced with a random hex value.*
sample_name		My Profile	The name of this profile (used in the Indicators of Compromise report)

Option	Context	Default Value	Changes
sleep			Default sleep time defined as either: seconds jitter (e.g. '20 25') or [n]d [n]h [n]m [n]s [n]j (e.g. '1d 13h 34m 45s 25j') This property cannot be used when the sleeptime and jitter options are included in the profile.
sleeptime		60000	Default sleep time (in milliseconds). This property cannot be used when the sleep option is included in the profile.
smb_frame_header			Prepend header to SMB Beacon messages
ssh_banner		Cobalt Strike 4.2	SSH client banner
ssh_pipename		postex_ssh_####	Name of pipe for SSH sessions. *Each # is replaced with a random hex value.*

Option	Context	Default Value	Changes
steal_token_access_mask		Blank/0 (TOKEN_ALL_ACCESS)	Sets the default used by steal_token beacon command and bsteal_token beacon aggressor script command for the OpenProcessToken functions "Desired Access". Suggestion: use "11" for "TOKEN_DUPLICATE | TOKEN_ASSIGN_PRIMARY | TOKEN_QUERY"
tasks_max_size		1048576	The maximum size (in bytes) of task(s) and proxy data that can be transferred through a communication channel at a check in
tasks_proxy_max_size		921600	The maximum size (in bytes) of proxy data to transfer via the communication channel at a check in.
tasks_dns_proxy_max_size		71680	The maximum size (in bytes) of proxy data to transfer via the DNS communication channel at a check in.
tcp_frame_header			Prepend header to TCP Beacon messages

Option	Context	Default Value	Changes
tcp_port		4444	Default TCP Beacon listen port
uri	http-get, http-post	[required option]	Transaction URI
uri_x86	http-stager		x86 payload stage URI
uri_x64	http-stager		x64 payload stage URI
useragent		Internet Explorer (Random)	Default User-Agent for HTTP comms.
verb	http-get, http-post	GET, POST	HTTP Verb to use for transaction
client_max_post_get_packet	http-post	4096 - 65536 Default: 4096	Controls the amount of file download data that a beacon can process during one cycle (check-in) when using HTTP Posts with the GET verb.
client_max_post_get_size	http-post	96 - 4096 Default: 96	Controls the size of chunked data (data sent per request) when beacon is posting with a GET verb into an HTTP Header.
client_max_post_post_size	http-post	1024 - 524288 Default: 524288	Chunks HTTP posted data (with the POST verb) into multiple smaller requests.

With the uri option, you may specify multiple URIs as a space separated string. Cobalt Strike's web server will bind all of these URIs and it will assign one of these URIs to each Beacon host when the Beacon stage is built.

Even though the useragent option exists; you may use the header statement to override this option.

Additional Considerations for the 'task_' Settings

The tasks_max_size, tasks_proxy_max_size,and tasks_dns_proxy_max_size work together to create a data buffer to be transferred to beacon when a check in occurs. When the beacon checks in it requests a list of tasks and proxy data that is ready to be transferred to this beacon and its children. The data buffer starts to fill with task(s) followed by proxy data for the parent beacon. Then it continues this pattern for each child beacon until no more tasks or proxy data is available or the tasks_max_size setting will be exceeded by the next task or proxy data.

The tasks_max_size controls the maximum size in bytes a data buffer filled with tasks and proxy data can be to transfer it to beacon through DNS, HTTP, HTTPS, and Peer-to-Peer communication channels. Most of the time the defaults are fine, however there are occasions when a cus*tom task will ex*ceed the maximum size and cannot be sent. For example, you use the execute-assembly with an executable larger than 1MB in size and the following message is displayed in the team server and beacon consoles.

```
[TeamServer Console]
Dropping task for 40147050! Task size of 1389584 bytes is over the max task size limit of 1048576 bytes.

[Beacon Console]
Task size of 1389584 bytes is over the max task size limit of 1048576 bytes.
```

Increasing the tasks_max_size setting will allow this custom task to be sent. However, it will require restarting the team server and generating new beacons as the tasks_max_size is patched into the configuration settings when a beacon is generated and cannot be modified. This setting also affects how much heap memory beacon allocates to process tasks.

Best Practices:

- Determine the largest task size that will be sent to a beacon. This can be done through testing and looking for the message above or investigating your custom objects (executables, dlls, etc) that are used in your engagements. Once this is determined add some extra space to the value. Using the information from the above example use 1572864 (1.5 MB) as the tasks_max_size. The reason to have extra space is because a smaller task may follow the larger task to read the response.
- When the tasks_max_size value is determined update the task_max_size setting in your profile and start the team server and generate your beacon artifacts to deploy on your target systems.

- If your infrastructure requires beacons generated from other team servers to connect with each other through Peer-to-Peer communication channels, then this setting should be updated on all team servers. Otherwise, a beacon will ignore a request when it exceeds its configured size.
- If you are using an ExternaC2 listener an update would be required to support tasks_max_size larger than the default size of 1MB.

When executing a large task avoid queueing it with other tasks, especially if this is being executed on a beacon using peer-to-peer communication channels (SMB and TCP) as it could be delayed for several check ins depending on the number of already queued tasks and proxy data to send. The reason is when a task is added it has a size of X bytes which reduces the total available space available for adding additional tasks. In addition, proxying data through a beacon will also reduce the amount of available space for sending a large task. When a task is delayed the following message is displayed in the team server and beacon consoles.

```
[Team Server Console]
Chunking tasks for 123! Unable to add task of 787984 bytes as it is over the available size of 260486 bytes. 2 task(s) on hold until next checkin.

[Beacon Console]
Unable to add task of 787984 bytes as it is over the available size of 260486 bytes. 2 task(s) on hold until next checkin.
```

The tasks_dns_proxy_max_size (DNS channel) and tasks_proxy_max_size (Other channels) controls the size of proxy data in bytes to be sent to beacon. Both settings need to be less than the tasks_max_size setting. It is recommended not to modify these settings as the default sizes are fine. How these settings work is when it is time to add proxy data to the data buffer for a parent beacon it uses the channels proxy_max_size setting minus the current task length, which can be either a positive or negative value. If it is a positive value, then the proxy data will be added up to that value. if it is a negative value the proxy data is skipped for this check in. For a child beacon the proxy_max_size is temporarily reduced based on the available data buffer space left from processing the parent and prior children.

RELATED TOPICS

C2Profile grammar

`dissect.cobaltstrike` utilizes the Lark parser for parsing and generating Cobalt Strike Malleable C2 Profiles.

The Lark grammar file to parse the *Profile Language* is defined in `c2profile.lark` and listed below for reference.

> **Note**
>
> Currently, the grammar implementation is pretty naive and could be improved upon. For example, the values are all *STRING* but could benefit from other types as well.

```
start: value*

?value: "set" OPTION string ";"                                          -> option
    | "http-config" "{" http_config_options* "}"                         -> http_config
    | "https-certificate" variant? "{" https_certificate_options* "}"    -> https_certificate
    | "code-signer" "{" code_signer_options* "}"                         -> code_signer
    | "http-stager" variant? "{" http_stager_options* "}"                -> http_stager
    | "http-get" variant? "{" http_get_options* "}"                      -> http_get
    | "http-post" variant? "{" http_post_options* "}"                    -> http_post
    | "stage" "{" stage_options* "}"                                     -> stage
    | "process-inject" "{" process_inject_options* "}"                   -> process_inject
    | "post-ex" "{" postex_options* "}"                                  -> post_ex
    | "dns-beacon" "{" dns_beacon_options* "}"                           -> dns_beacon
    | "http-beacon" "{" http_beacon_options* "}"                         -> http_beacon

OPTION: "sample_name"
    | "data_jitter"
    | "dns_idle"
    | "dns_max_txt"
    | "dns_sleep"
    | "dns_stager_prepend"
    | "dns_stager_subhost"
    | "dns_ttl"
    | "host_stage"
    | "jitter"
    | "maxdns"
    | "pipename"
    | "pipename_stager"
    | "sleeptime"
    | "smb_frame_header"
    | "ssh_banner"
    | "ssh_pipename"
    | "tcp_frame_header"
    | "tcp_port"
    | "useragent"
    | "spawnto"                // deprecated since Cobalt Strike 3.6
    | "spawnto_x86"            // moved to post-ex since Cobalt Strike 3.14
    | "spawnto_x64"            // moved to post-ex since Cobalt Strike 3.14
    | "amsi_disable"           // moved to post-ex since Cobalt Strike 3.14
    | "create_remote_thread"   // deprecated since Cobalt Strike 3.12
    | "hijack_remote_thread"   // deprecated since Cobalt Strike 3.12
    | "tasks_max_size"         // introduced in Cobalt Strike 4.6
    | "tasks_proxy_max_size"   // introduced in Cobalt Strike 4.6
    | "tasks_dns_proxy_max_size"// introduced in Cobalt Strike 4.6

http_config_options: "set" "headers" string ";"     -> headers
```

```
    | "header" string string ";"                    -> header
    | "set" "trust_x_forwarded_for" string ";"      -> trust_x_forwarded_for
    | "set" "block_useragents" string ";"           -> block_useragents
    | "set" "allow_useragents" string ";"           -> allow_useragents

http_stager_options: "set" "uri_x86" string ";"     -> uri_x86
    | "set" "uri_x64" string ";"                    -> uri_x64
    | "client" "{" http_options* "}"                -> client
    | "server" "{" http_options* "}"                -> server

http_options: "header" string string ";"            -> header
    | "parameter" string string ";"                 -> parameter
    | "output" "{" data_transform* "}"              -> output

data_transform: steps termination

steps: transform_statement*
termination: termination_statement ~ 1

transform_statement: "append" string ";"    -> append
    | "base64" ";"                          -> base64
    | "base64url" ";"                       -> base64url
    | "mask" ";"                            -> mask
    | "netbios" ";"                         -> netbios
    | "netbiosu" ";"                        -> netbiosu
    | "prepend" string ";"                  -> prepend

termination_statement: "header" string ";"  -> header
    | "parameter" string ";"                -> parameter
    | "print" ";"                           -> print
    | "uri-append" ";"                      -> uri_append

stage_transform: "prepend" string ";"       -> prepend
    | "append" string ";"                   -> append
    | "strrep" string string ";"            -> strrep

http_get_options: "set" "uri" string ";"            -> uri
    | "set" "verb" string ";"                       -> verb
    | "client" "{" http_get_client_options* "}"     -> client
    | "server" "{" http_options* "}"                -> server

http_get_client_options: "header" string string ";" -> header
    | "set" "verb" string ";"                       -> verb
    | "metadata" "{" data_transform* "}"            -> metadata
    | "id" "{" data_transform*  "}"                 -> id
    | "parameter" string string ";"                 -> parameter
    | "output" "{"  data_transform*  "}"            -> output
```

```
http_post_options: "set" "uri" string ";"           -> uri
    | "set" "verb" string ";"                       -> verb
    | "client" "{" http_get_client_options* "}"     -> client
    | "server" "{" http_options* "}"                -> server

https_certificate_options: "set" "C" string ";"     -> country
    | "set" "CN" string ";"                         -> common_name
    | "set" "L" string ";"                          -> locality
    | "set" "OU" string ";"                         -> org_unit
    | "set" "O" string ";"                          -> org
    | "set" "ST" string ";"                         -> state
    | "set" "validity" string ";"                   -> validity
    | "set" "keystore" string ";"                   -> keystore
    | "set" "password" string ";"                   -> password

code_signer_options: "set" "keystore" string ";"    -> keystore
    | "set" "password" string ";"                   -> password
    | "set" "alias" string ";"                      -> alias
    | "set" "digest_algorithm" string ";"           -> digest_algorithm
    | "set" "timestamp" string ";"                  -> timestamp
    | "set" "timestamp_url" string ";"              -> timestamp_url

stage_options: "string" string ";"                  -> string
    | "stringw" string ";"                          -> stringw
    | "transform-x86" "{" stage_transform* "}"      -> transform_x86
    | "transform-x64" "{" stage_transform* "}"      -> transform_x64
    | "set" "allocator" string ";"                  -> allocator
    | "set" "cleanup" string ";"                    -> cleanup
    | "set" "magic_pe" string ";"                   -> magic_pe
    | "set" "magic_mz_x86" string ";"               -> magic_mz_x86
    | "set" "magic_mz_x64" string ";"               -> magic_mz_x64
    | "set" "obfuscate" string ";"                  -> obfuscate
    | "set" "sleep_mask" string ";"                 -> sleep_mask
    | "set" "smartinject" string ";"                -> smartinject
    | "set" "stomppe" string ";"                    -> stomppe
    | "set" "userwx" string ";"                     -> userwx
    | "set" "compile_time" string ";"               -> compile_time
    | "set" "entry_point" string ";"                -> entry_point
    | "set" "module_x86" string ";"                 -> module_x86
    | "set" "module_x64" string ";"                 -> module_x86
    | "set" "image_size_x86" string ";"             -> image_size_x86
    | "set" "image_size_x64" string ";"             -> image_size_x64
    | "set" "name" string ";"                       -> name
    | "set" "rich_header" string ";"                -> rich_header
    | "set" "checksum" string ";"                   -> checksum
    | "set" "syscall_method" string ";"             -> syscall_method          // introduced in
```

```
Cobalt Strike 4.8
    | "set" "data_store_size" string ";"            -> data_store_size          // introduced in
Cobalt Strike 4.9
    | "beacon_gate" "{" beacon_gate_options* "}"    -> beacon_gate              // introduced in
Cobalt Strike 4.10

process_inject_options: "set" "allocator" string ";"    -> allocator
    | "set" "min_alloc" string ";"                      -> min_alloc
    | "set" "startrwx" string ";"                       -> startrwx
    | "set" "userwx" string ";"                         -> userwx
    | "transform-x86" "{" stage_transform* "}"          -> transform_x86
    | "transform-x64" "{" stage_transform* "}"          -> transform_x64
    | "execute" "{" execute_options* "}"                -> execute
    | "disable" string ";"                              -> disable
    | "set" "bof_allocator" string ";"                  -> bof_allocator        // introduced in
Cobalt Strike 4.7
    | "set" "bof_reuse_memory" string ";"               -> bof_reuse_memory     // introduced in
Cobalt Strike 4.7

execute_options: "CreateThread" string ";"  -> createthread_special
    | "CreateRemoteThread" string ";"       -> createremotethread_special
    | "CreateThread" ";"                    -> createthread
    | "CreateRemoteThread" ";"              -> createremotethread
    | "NtQueueApcThread" ";"                -> ntqueueapcthread
    | "NtQueueApcThread-s" ";"              -> ntqueueapcthread_s
    | "RtlCreateUserThread" ";"             -> rtlcreateuserthread
    | "SetThreadContext" ";"                -> setthreadcontext

beacon_gate_options: "None" ";"             -> none
    | "Comms" ";"                           -> comms
    | "Core" ";"                            -> core
    | "Cleanup" ";"                         -> cleanup
    | "All" ";"                             -> all
    | "InternetOpenA" ";"                   -> internetopena
    | "InternetConnectA" ";"                -> internetconnecta
    | "VirtualAlloc" ";"                    -> virtualalloc
    | "VirtualAllocEx" ";"                  -> virtualallocex
    | "VirtualProtect" ";"                  -> virtualprotect
    | "VirtualProtectEx" ";"                -> virtualprotextex
    | "VirtualFree" ";"                     -> virtualfree
    | "GetThreadContext" ";"                -> getthreadcontext
    | "SetThreadContext" ";"                -> setthreadcontext
    | "ResumeThread" ";"                    -> resumethread
    | "CreateThread" ";"                    -> createthread
    | "CreateRemoteThread" ";"              -> createremotethread
    | "OpenProcess" ";"                     -> openprocess
    | "OpenThread" ";"                      -> openthread
```

```
    | "CloseHandle" ";"                    -> closehandle
    | "CreateFileMappingA" ";"             -> createfilemappinga
    | "MapViewOfFile" ";"                  -> mapviewoffile
    | "UnmapViewOfFile" ";"                -> unmapviewoffile
    | "VirtualQuery" ";"                   -> virtualquery
    | "DuplicateHandle" ";"                -> duplicatehandle
    | "ReadProcessMemory" ";"              -> readprocessmemory
    | "WriteProcessMemory" ";"             -> writeprocessmemory
    | "ExitThread" ";"                     -> exitthread

postex_options: "set" "spawnto_x86" string ";"  -> spawnto_x86
    | "set" "spawnto_x64" string ";"            -> spawnto_x64
    | "set" "obfuscate" string ";"              -> obfuscate
    | "set" "pipename" string ";"               -> pipename
    | "set" "smartinject" string ";"            -> smartinject
    | "set" "amsi_disable" string ";"           -> amsi_disable
    | "set" "keylogger" string ";"              -> keylogger
    | "set" "thread_hint" string ";"            -> thread_hint

dns_beacon_options: "set" "dns_idle" string ";" -> dns_idle
    | "set" "dns_max_txt" string ";"            -> dns_max_txt
    | "set" "dns_sleep" string ";"              -> dns_sleep
    | "set" "dns_ttl" string ";"                -> dns_ttl
    | "set" "maxdns" string ";"                 -> maxdns
    | "set" "dns_stager_prepend" string ";"     -> dns_stager_prepend
    | "set" "dns_stager_subhost" string ";"     -> dns_stager_subhost
    | "set" "beacon" string ";"                 -> beacon
    | "set" "get_A" string ";"                  -> get_a
    | "set" "get_AAAA" string ";"               -> get_aaaa
    | "set" "get_TXT" string ";"                -> get_txt
    | "set" "put_metadata" string ";"           -> put_metadata
    | "set" "put_output" string ";"             -> put_output
    | "set" "ns_response" string ";"            -> ns_response
    | "#" "dns_resolver" string ";"             -> comment_dns_resolver

http_beacon_options: "set" "library" string ";" -> library              // introduced in Cobalt
Strike 4.9
    | "set" "data_required" string ";"          -> data_required        // introduced in Cobalt
Strike 4.10
    | "set" "data_required_length" string ";"   -> data_required_length // introduced in Cobalt
Strike 4.10

header: string
string: STRING
variant: string

STRING: "\"" /(.|\n)*?/ /(?<!\\)(\\\\)*?/ "\""
```

```
%import common.WS
%import common.SH_COMMENT
%import common.NEWLINE

%ignore WS
%ignore SH_COMMENT
%ignore NEWLINE
```